NumPy Tutorial 6: Mathematical and Statistical Functions

This chapter covers the built in mathematical and statistical functions in NumPy. After completing this tutorial, students will understand how to use NumPy for data analysis using functions like mean, median, standard deviation, and more.

---

1. Introduction

In previous tutorials, we learned basic operations on arrays. In this tutorial, we will go deeper into NumPy's powerful mathematical and statistical functions.

These functions are used every day in Data Science and Exploratory Data Analysis. Understanding them properly is very important for analyzing real world datasets.

---

2. Basic Statistical Functions

Let us start with a simple dataset and apply statistical functions.

```
import numpy as np

marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

print("Sum:", np.sum(marks))
print("Minimum:", np.min(marks))
print("Maximum:", np.max(marks))
print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Standard Deviation:", np.std(marks))
print("Variance:", np.var(marks))
```

Output:

```
Sum: 699
Minimum: 45
Maximum: 95
Mean: 69.9
Median: 72.0
Standard Deviation: 16.42
Variance: 269.69
```

Function	Description
np.sum	Total of all values
np.min	Smallest value
np.max	Largest value
np.mean	Average of all values
np.median	Middle value when sorted
np.std	How spread out the values are
np.var	Square of standard deviation

---

3. **Understanding Mean, Median, and Standard Deviation**

Mean is the average. Add all values and divide by count.

Median is the middle value when all values are sorted. It is not affected by very large or very small values.

Standard Deviation tells us how far values are from the mean. A small std means values are close together. A large std means values are spread out.

Example:

```
data1 = np.array([48, 50, 51, 49, 52])
data2 = np.array([10, 90, 5, 95, 50])

print("Data1 Std:", np.std(data1))
print("Data2 Std:", np.std(data2))
```

Output:

```
Data1 Std: 1.41
Data2 Std: 35.3
```

data1 has a small std — values are close together. data2 has a large std — values are very spread out.

---

4. **Finding Index of Min and Max**

Sometimes we need to find the position of the minimum or maximum value.

```
marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

print("Index of Min:", np.argmin(marks))
print("Index of Max:", np.argmax(marks))
```

Output:

```
Index of Min: 4
Index of Max: 8
```

- argmin returns the index of the smallest value — 45 is at index 4.
- argmax returns the index of the largest value — 95 is at index 8.

---

5. **Sorting Arrays**

NumPy provides a simple way to sort arrays.

```
arr = np.array([30, 10, 50, 20, 40])

print("Sorted:", np.sort(arr))
print("Original:", arr)
```

Output:

```
Sorted: [10 20 30 40 50]
Original: [30 10 50 20 40]
```

np.sort does not change the original array — it returns a new sorted array.

Sorting in Descending Order:

```
print(np.sort(arr)[::-1])
```

Output:

```
[50 40 30 20 10]
```

---

6. **Statistical Functions on 2D Arrays**

For 2D arrays, we can calculate statistics along rows or columns.

```
import numpy as np

scores = np.array([[85, 90, 78],
                   [92, 88, 95],
                   [70, 75, 80]])

print("Column means:", np.mean(scores, axis=0))
print("Row means:", np.mean(scores, axis=1))
```

Output:

```
Column means: [82.33 84.33 84.33]
Row means: [84.33 91.67 75.0]
```

- axis=0 — calculates along columns (result is per column).
- axis=1 — calculates along rows (result is per row).

---

7. **Cumulative Functions**

Cumulative functions calculate a running total.

Cumulative Sum

```
arr = np.array([1, 2, 3, 4, 5])
print(np.cumsum(arr))
```

Output:

```
[ 1  3  6 10 15]
```

Each value is the sum of all previous values. 1, 1+2=3, 3+3=6, 6+4=10, 10+5=15.

Cumulative Product

```
print(np.cumprod(arr))
```

Output:

```
[  1   2   6  24 120]
```

---

8. **Rounding Functions**

```
arr = np.array([1.234, 2.567, 3.891, 4.123])

print(np.round(arr, 2))
print(np.floor(arr))
print(np.ceil(arr))
```

Output:

```
[1.23 2.57 3.89 4.12]
[1. 2. 3. 4.]
[2. 3. 4. 5.]
```

- round — rounds to specified decimal places.
- floor — always rounds down.
- ceil — always rounds up.

---

9. **Percentile and Quantile**

Percentile tells us the value below which a given percentage of data falls.

```
marks = np.array([55, 72, 68, 90, 45, 88, 76, 60, 95, 50])

print("25th percentile:", np.percentile(marks, 25))
print("50th percentile:", np.percentile(marks, 50))
print("75th percentile:", np.percentile(marks, 75))
```

Output:

```
25th percentile: 57.5
50th percentile: 72.0
75th percentile: 85.0
```

These three values are also called Q1, Q2, and Q3 — very commonly used in Exploratory Data Analysis.

---

10. **Correlation and Covariance**

These functions measure the relationship between two arrays.

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

print("Correlation:\n", np.corrcoef(x, y))
print("Covariance:\n", np.cov(x, y))
```

- Correlation close to 1 means strong positive relationship.
- Correlation close to -1 means strong negative relationship.
- Correlation close to 0 means no relationship.

---

11. **Conclusion of Tutorial 6**

In this tutorial, students have learned:

1. How to use basic statistical functions — sum, min, max, mean, median, std.
2. The difference between mean, median, and standard deviation.
3. How to find the index of minimum and maximum values.
4. How to sort arrays.
5. How to apply statistical functions on 2D arrays using axis.
6. How cumulative sum and product work.
7. How to use rounding functions.
8. What percentile and quartiles are.
9. How correlation and covariance work.

In the next tutorial, we will learn about Boolean indexing — filtering data using conditions.

This completes NumPy Tutorial 6.